

Information Disclosure- Access Control and Authentication labs

7BUIS022W: Class activities

Dr Ayman El hajjar

Week 4-5

Lab Objectives

- **You are expected to work on this lab for two weeks (week 4 and Week 5) in class**
 1. Information Disclosure
 2. Access Control
 3. Authentication
- In class, If you are using a university machine: You only need Burp Suite Professional (already installed) and OWASP VM set on "Host Only Adapter"
- If you are using your own machine: You will need Burp Suite community edition Installed on your machine or on Kali Linux VM (already installed) and OWASP VM set on "Host Only Adapter"
- In this lab, we will be attempt to exploit OWASP using the vulnerabilities you learn in the previous lab.

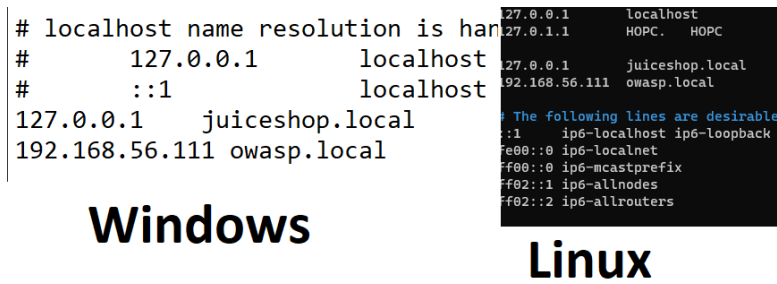


Figure 1: Hosts file configuration on Windows or Linux

Note: OWASP IP address

- **MY IP address is 192.168.56.111**
- However on my windows machine, for ease of practice, I have changed the file **hosts** to point 192.168.56.111 to owasp.local as shown in Figure 1
- On windows, you can find the hosts file in C:\Windows\System32\drivers\etc
- You can do the same if you are on Linux or Kali. The location of the **hosts** file in Linux is in /etc folder.

1 Information Disclosure

- Before starting the labs:
 - You should have read about Information disclosure vulnerabilities and completed the required activities on this link [Burp Suite Information disclosure vulnerabilities page](#)
- For each of the activities below, you should first complete it on BurpSuite Academy.
- Complete the following activities on OWASP:
 1. Information disclosure in error messages ☐
 - With Burp Suite running, we will attempt to modify the parameters on the ProductID on the BodgeIt store and enter instead of an Integer (what the server expects), any String of text, We will observe how BodgeIt behaves.
 - You can find the bodgeIt store link on the OWASP main server page or you can use this link <http://owasp.local/bodgeit/>

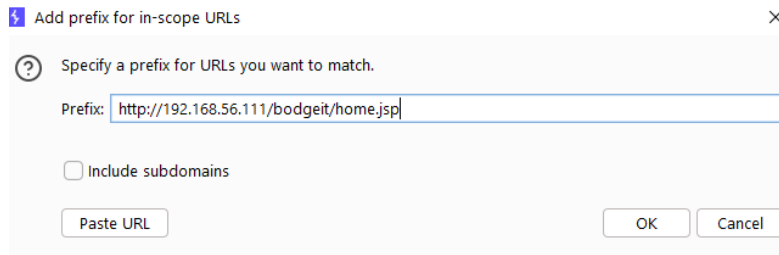


Figure 2: BodgeIt in the scope

- * Does the server behave as expected to the user, and return an error that this product does not exist?
- * Does it reveals much about the what kind of Back End information the store is built on?

2. Information disclosure on debug page ☐

- With Burp Suite Professional running, we will try to look for comments in URL paths for reference to comments or scripts. The developers put these comments for future reference when developing or for debugging. Sometimes they don't remove them at all. You might be able to find important debug information or maybe passwords that are hardcoded.
- **Note: If you find there are many sites on Burpsuite from previous activity, you can close BurpSuite and start a new project**
- Click on Dashboard tab, and select "**New Live Task**"
- On the "**New Live Task**" window, select "**Live passive crawl**"
- On the URL scope, select "**Suite Scope**" and press **OK**.
- Now select the **Target** tab and click on **Scope Settings**
- Add the BodgeIt home page link as in the Figure 2.
- Go back to your browser and point it to Bodgeit homepage
`http://owasp.local/bodgeit/`
- After BurpSuite crawl Bodgeit store, while on the Target tab, find the Bodgeit store folder in the sitemap tab
- Right click on the folder **Bodgeit** and select **Engagement tools** then **Find Comments**
- Observe the files URLs found.
- Note the URL path to a file called "admin.jsp"
- Explore this file
 - * What information this file revealed?

3. Source code disclosure via backup files ☐

- With Burp Suite Professional running, we will try to look for hidden folders and files. We might find something of Interest that the developers did not want a search Engine to crawl. You might find a lot of information hidden in those folders and files.
- If you find there are many sites on Burpsuite from previous activity, you can close BurpSuite and start a new project
- Click on Dashboard tab, and select "New Live Task"
- On the "New Live Task" window, select "Live passive crawl"
- On the URL scope, select "Suite Scope" and press OK.
- Now select the Target tab and click on Scope Settings
- Go to your browser and point it to bWAPP homepage:for me it is <http://owasp.local/bWAPP/>
- Add the bWAPP home page link <http://owasp.local/bWAPP/>
- On Burpsuite, while on the Target tab
- Right click on the folder **bWAPP** and select **Passively Scan this branch tools**
- Observe the files URLs found.
- In there any backup files? (Hint Explore the Issues information on the Target tab) as in Figure 3

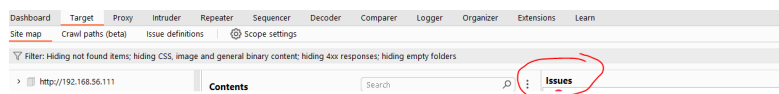


Figure 3: Issues on Target tab

4. Authentication bypass via information disclosure ☐

- With Burp Suite Professional running, we will capture a user logging in with low privilege.
- We will then attempt to escalate the privilege.
- Click on Dashboard tab, and select "New Live Task"
- On the "New Live Task" window, select "Live passive crawl"
- On the URL scope, select "Suite Scope" and press OK.
- Now select the Target tab and click on Scope Settings
- Go to your browser and point it to Mutillidae homepage <http://owasp.local/Mutillidae/>
- Click on the Login/Register button

- Log in using username **user** and password **user**
- On Burpsuite, while on the Target tab
- Find the response to the POST message (where you entered your username)
- Send the response to the repeater
- In the repeater Request windows , see what information can be useful.
- I cannot see the authorization header, which means it is hidden.
- However i did not get an error. That tells me that the user admin does exist!



Figure 4: User To Admin

- Explore other OWASP applications and tools, see if you can find anything interesting

2 Access Control

- Before starting the labs:
 - Read about Access control vulnerabilities and privilege escalation
- For each of the activities below, you should first complete it on BurpSuite Academy.
- Complete the following activities on OWASP:

1. Unprotected admin functionality with unpredictable URL ☐
 - This lab is quite straight forward, and does not require BurpSuite. We want to see if the robots.txt file gives us some indication of how Admins access the page.
 - * Read about robots.txt on this link
 - * Many websites provide files at `/robots.txt` and `/sitemap.xml` to help crawlers navigate their site. Among other things, these files often list specific directories that the crawlers should skip, for example, because they may contain sensitive information.

- * As these files are not usually linked from within the website, they may not immediately appear in Burp's site map. However, it is worth trying to navigate to /robots.txt or /sitemap.xml manually to see if you find anything of use. "
 - Point your browser to The Mutillidae application
http://owasp.local/mutillidae
 - Now add the robots.txt to the address as following
http://owasp.local/mutillidae/robots.txt
 - Observe what you find. Try pointing your browser to those folders.
 - * What information this file revealed? Try to explore those information and see what else you can find.
 - * Does any of the "Disallowed" folders and paths gives you access to admin process!
 - Now point your browser to The bWAPP application
http://owasp.local/bWAPP
 - And add the robots.txt to the address as following
http://owasp.local/bWAPP/robots.txt
 - * What information this file revealed? Try to explore those information and see what else you can find.
 - * Does any of the "Disallowed" folders and paths gives you access to admin process!
 - Now you have explored the Robots.txt, attempt to solve the Failure to Restrict URL Access lesson and challenges in Shepherd Security.
 - * Can you see what a request parameter can leak within a cookie response?
 - * Can you see what information you can obtain by simply reading the code?
 - * Can you solve this lesson?
 - * Can you see how you can escalate your privilege?
2. Unprotected admin functionality with unpredictable URL ☐
- There are no admin functionalities embedded within Java scripts (to my knowledge) however I would recommend that you explore the different OWASP tools source codes and look specifically at the embedded Java scripts . It might gives a lot more information that you might think!
 - I suggest you start with the Ghost application and the Bricks login page number 2

3. The activities below are related to several labs:
 - The idea of the activity is to identify potential parameters that can be modified on the user side, and more precisely parameters stored in cookies that can be modified on the user side and forces the website to interact with such changes.
 - In the PortSwigger lab that you have completed, you should have been able to change parameters for cookies stored about the access and elevate the user privilege to admin.
 - Try to login to different applications on OWASP and observe which reveals information about session parameters in the URL as in User role controlled by request parameter ☐.
4. User ID controlled by request parameter with data leakage in redirect ☐
 - Attempt to solve the Broken Session Management lesson in Shepherd Security.
 - * Can you see what a request parameter can leak within a cookie response?
 - * Can you solve this lesson?
5. User ID controlled by request parameter with password disclosure ☐
 - All labs about what type of data leaks and controlled by the request parameter, can be obtained by repeating the traffic through burpsuite using a different session ID.
 - This can be by repeating a user ID in a URL or a comment or session ID appearing in the URL as in User ID controlled by request parameter ☐
 - or Repeating a random session ID and predict what happens in User ID controlled by request parameter, with unpredictable user IDs ☐
6. Let's Try one application that reveals some information in the URL about the User parameters.
 - Point your browser to The WackPicko application
`http://owasp.local/WackPicko`
 - Login with a normal username. You can find the login details for all applications by going to the OWASP homepage and looking at the source code.
 - Observe what you find in the URL. Look specifically at the user ID.
 - * What information this file revealed?
 - * Can you see other users accounts contents?

7. User role can be modified in user profile ☐
8. Insecure direct object references ☐
 - Login to Security Shepherd.
 - Click on Challenges
 - Select "Insecure Direct Object Reference challenge 1."
 - * Attempt to solve this challenge. Hint (Not all users exist in the list)
 - Select "Insecure Direct Object Reference challenge 2."
 - * Attempt to solve this challenge. Hint (User ID are encrypted using MD5 encryption. Try to decrypt them using MD5 encryption/Decryption tool)
- There are many labs for access controls that will provide a challenge for you on OWASP. Try to conduct the ones on " WebGoat: Access Control Flaws

3 Authentication

- Read about Authentication vulnerabilities
- For each of the activities below, you should first complete it on BurpSuite Academy.
- OWASP is not vulnerable to all the Authentication vulnerabilities that were completed on BurpSuite Academy
- Complete the following activities on OWASP:
 1. Username enumeration via different responses ☐
 - In this lab, we will attempt to brute force both usernames and passwords using dictionaries and look for the HTTP response for each entry.
 - * To demonstrate this activity, we will be using OrangeHRM on OWASP. **However this activity will attempt to break all tools and application on the OWASP VM**
 - * Start BurpSuite and OWASP VM.
 - While BurpSuite is intercepting traffic
 - Browse to orangeHRM
http://owasp.local/orangehrm/login.php
 - Now to login, try any combination of usernames and password, even if they dont exist.
 - On Burpsuite- click on the proxy tab on and select HTTP history
 - You should see the HTTP post login message

- Right click on it and select **Send to intruder**
- Now click on the **Intruder** tab (This will be done twice, once for usernames and once for passwords) (see Fig5)
- * Make sure that the attack type selected is **Sniper**
 - 1- Highlight the Username or the password
 - 2- Select **Add \$**
 - 3- Select **Payloads** tab
 - 4- Paste either the Username list from users.txt or the passwords list from passwords.txt depending on which value you highlighted in the first step. Both files are provided on Blackboard
 - 5- Click **Start Attack**

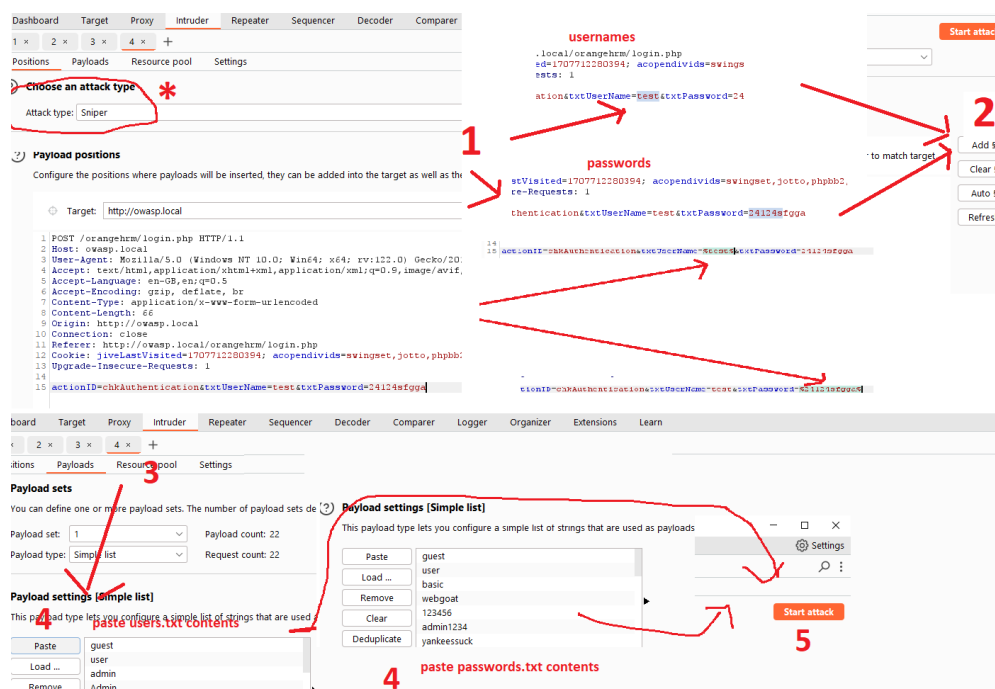


Figure 5: Payloads for Brute Force

- You will have to do the same activities for the passwords as well.
 - * Although all requests responds with the Status code 200 because the server still responds even it the username or password are wrong. It will responds to tell the user that the username is wrong or the password is wrong.
 - * However you will notice that the responses length varies. A longer response means that this value was successful. you will find that

Several usernames are valid. You can try them to see which one works.

2. Password reset broken logic ☐

- On the OWASP VM, go to webgoat
- Click on Authentication flaws and select "Forgot Password"
- Make sure Burpsuite is running
- **Now although on owasp there are no email client and user should answer certain questions, the outcome of the lab is the same.**
- Enter webgoat as the username, and the answer to the favourite colour is red.
- Observer the traffic.
- Now lets try with another user: admin
- We obviously dont know the password.
- Now let's click on forgot password try with another color, (not red). You will notice that the system does not employ any method to lock the account after several attempts.
- Now using the same method we used in the first authentication activity 1, create a small payload of various colors and attempt to get the correct answer. Explore the results and see if you successfully managed to obtain the admin password.

3. Username enumeration via subtly different responses ☐

4. Username enumeration via response timing ☐

5. Broken brute-force protection, IP block ☐

6. 2FA simple bypass ☐

- On the OWASP VM, go to webgoat
- Click on Authentication flaws and select "Multi Level login 2".
- Start BurpSuite and make sure that the Proxy is set to Intercept traffic
- Login with the username and password provided
- Enter the Tan value for the number you were given
- Before you forward the submit, change Joe to Jane.
- You now have access to details for Jane Credit Card

7. Brute-forcing a stay-logged-in cookie ☐

- On the OWASP VM, go to webgoat
- Click on Session Management Flaws flaws and select "Spoof an Authentication Cookie".

– attempt to solve it using similar techniques to the BurpsUite academy

8. Password brute-force via password change ☐

- Now you have completed the labs, in the class you will be looking to identify if those vulnerabilities exist on certain applications on OWASP. More information will be provided in the "In Class Lab document lab"
- You are also expected to attempt to complete similar activities on your assignment scenario. Note, you will not succeed in all attempts.